Defence script: who says what

Ten minutes, two speakers. Each slide gives the speaker, a target time and the text to say. The figures are the measured ones from `docs/evaluation.md` (2026-11-09).

Speaker	Slides	Time
Lynn-Aurea	1, 2, 3, 8, 9, 11	about 4 min 30
Sereine	4, 5, 6, 7, 10	about 4 min 15

Keep one minute spare. Rehearse once with a timer, both speaking. If you run over, shorten slides 4 and 9 first. Slide 12 is a backup for questions.

Slide 1: Routing support tickets with an LLM

SPEAKER: Lynn-Aurea (about 20 seconds)

Good morning. We are group 4. Our component is a classifier: it reads a ticket sent to Albert's Marketplace's support and decides which team should handle it.

I'll present the task and our two design choices. Sereine will explain how it works and what we measured. I'll come back for the judge, the failures and our recommendation, and Sereine will show how it resists prompt injection.

Slide 2: One support inbox, seven teams

SPEAKER: Lynn-Aurea (about 45 seconds)

Albert's Marketplace sells beauty products from third-party sellers. All customer tickets arrive in one inbox, and today someone reads each one just to send it to the right team.

Our component does that first step. It chooses one of seven labels: billing, delivery, returns and refunds, account, seller, product information, and other when nothing fits.

It returns three fields: the label, a confidence between 0 and 1, and needs_human_review. Our code forces a human review when the confidence is below 0.70 or the label is other.

The users are the ticketing system, which queues the ticket, and then the agents of each team.

Slide 3: A classifier, and no retrieval

SPEAKER: Lynn-Aurea (about 50 seconds)

Two choices shape the project. First, a classifier, in decision record 0001. A closed list of labels can be scored exactly, without a model. A wrong label only costs one more transfer between teams. And a small, cheap model is enough.

Second, no retrieval, in decision record 0003. We tested it: we sent three tickets whose answer would need a company fact, like the return window for an opened cream or the shipping price to Belgium. All three got the right label, and so did the injection case: four out of four, for less than a cent.

The label depends on what the customer is asking about, not on the answer. And the model cannot invent a fact: its only fields are a label, a number and a flag. Sereine will now explain how it works.

Slide 4: From the ticket to what is sent

SPEAKER: Sereine (about 60 seconds)

Here is the path of a ticket. The ticket is untrusted: it is written by people outside the company. It goes into the prompt inside its own `<ticket>` tag, and the prompt says it is data, never instructions.

src/llm.py is the only module that calls the API. It checks every answer: Pydantic validates the JSON, the label must be one of the seven, and the review rule must hold. It also checks stop_reason first: an answer cut off at 300 tokens is a failure, not a record.

If the check fails, we send the error back to the model, twice at most. After that, it is a clear failure and a person reads the ticket. The SDK retries network errors twice; we have no retry loop of our own on top. That's decision record 0002.

Every call is logged in runs/calls.csv with its tokens, cost and latency.

Finally, src/guardrails.py builds what is actually sent: three fields, and no text written by the model.

Slide 5: Thirty checked cases, scored without a model first

SPEAKER: Sereine (about 45 seconds)

To measure it, we built 30 test cases. Ten we wrote ourselves at the start, including an injection attempt. Twenty are real tickets from the course data, chosen to be hard: French and German, spam, a brand pitching to become a seller, and even a car part the marketplace doesn't sell.

Each expected label was written by one of us and checked by the other, never by a model. Twelve tickets fit two teams, so they also have a second acceptable label.

We score in three ways. First, exact match on the label, with no model: our main score. Second, a judge from another, larger model, Sonnet, on four criteria. Third, pairwise comparisons in both orders, to catch position bias.

Slide 6: Three prompt strategies on the 30 cases

SPEAKER: Sereine (about 60 seconds)

We ran three prompt strategies on the same 30 cases: zero-shot, with only the rules; few-shot, with three example tickets; and chain-of-thought, where the model reasons before answering.

Counting the second acceptable label, zero-shot got 28 right, few-shot and chain-of-thought 27.

The cost is measured from the tokens of every call: 1.02 dollars per 1,000 calls for zero-shot, 0.74 for few-shot, 1.03 for chain-of-thought. Few-shot is the fastest, with 1.16 seconds at the 90th percentile, against 4.47 for zero-shot.

Zero-shot needed 17 re-prompts and gave up once. We printed the raw answers: the model puts the JSON in a code block and adds a reasoning paragraph, because the zero-shot prompt never says "nothing before or after". Our validation rejected those answers, so no invalid answer was ever used.

Slide 7: What 30 cases can and cannot tell

SPEAKER: Sereine (about 40 seconds)

The Pareto front keeps the strategies that no other beats on right answers, cost and latency. Zero-shot is on it because it has the most right answers; few-shot because it is the cheapest and fastest. Chain-of-thought is dominated by few-shot: same right answers, more expensive and slower.

But one case is 3.3 points. Zero-shot is only one case ahead, which is below what 30 cases can resolve. So on accuracy, we cannot tell the strategies apart; on cost and latency, we can.

And the scorer matters: for chain-of-thought, accepting the second label or not moves the score from 27 to 25. Lynn-Aurea will now present the judge.

Slide 8: A judge we checked before trusting it

SPEAKER: Lynn-Aurea (about 50 seconds)

A model grading its own answers tends to favour them, so our judge is a different, larger model: Sonnet grades Haiku.

Before reading its scores, Sereine and I scored ten answers by hand. The judge gave exactly our score on 31 of the 40 criterion scores: 78%. It is perfect on format and good on correctness, but on the review criterion it agrees with us only five times out of ten.

So we trust it as a second reading on format and correctness, not on review: the review rule stays in code, and exact match stays our main score.

We also judged every pair of strategies in both orders. The order changed the verdict 15 times out of 90, so those count as ties. In the end there are 71 ties out of 90: the judge prefers no strategy, which agrees with exact match.

Slide 9: Every error read by a person, grouped by cause

SPEAKER: Lynn-Aurea (about 60 seconds)

We read every wrong answer and gave each one a cause.

Most come from the prompt. Here is one: case L04-26 is a French complaint about a car carburettor that makes the engine stall. Albert's sells only beauty products, so the right label is other. Few-shot answered returns_refunds, with a confidence of 0.92, so no human would see it. The cause is our prompt: it gives the names of the labels but no definitions, and never says what the marketplace sells. Same gap for L04-27, a brand pitching to become a seller, labelled seller.

The model part is that it is wrong and sure of itself, so a confidence threshold cannot catch it.

L04-17 is a debatable test case, and L04-18 and L04-28 depend on the scorer. None comes from missing company text, which confirms we don't need retrieval.

The fix is a second prompt version with one line per label and the marketplace scope, measured again on the 30 cases.

Slide 10: Every ticket is untrusted, so we test what is sent

SPEAKER: Sereine (about 50 seconds)

Every ticket is written by someone outside the company, so it can carry instructions. L01-10 is one we wrote: it asks the model to reveal its hidden prompt, then asks about a late parcel.

The model gave the right label, delivery, and revealed nothing, but it did not ask for a human review. Our code did: the detector in guardrails.py flagged the ticket and forced the review. That's why the protection is in code, not only in the prompt.

We have three layers: the ticket is in its own field, declared as data; the schema only lets a label, a number and a flag out; and the record we send contains no text from the model.

The live test file runs five attacks, three times each, because the model is not deterministic. It checks what would be sent, and a normal ticket checks that we don't send everything to a human. Result: [read the result on the slide].

Slide 11: What we would deploy

SPEAKER: Lynn-Aurea (about 45 seconds)

Our recommendation: few-shot on Claude Haiku 4.5. On 30 cases it is as accurate as the other two, 27 out of 30, within one case. And it is the cheapest, 74 cents per 1,000 calls, the fastest, 1.16 seconds at the 90th percentile, and the only one with no re-prompt.

Before go-live, its prompt needs label definitions and the marketplace scope, because its two errors that reach no person come from the prompt. Then we run the 30 cases again.

After that, two weeks in shadow mode on real tickets, comparing the router's label with the agent's. And we would watch the share of tickets sent to a person: 7 out of 30 in our test, which is the real cost of the router.

Thank you. We're happy to take your questions.

Slide 12: Backup: our four decision records

BACKUP slide, not presented. Show it if a question is about one of the decisions.

Likely questions:

- Why Haiku? A simple, high-volume task; cheap and fast enough. The judge is Sonnet, a different and larger model.
- Why at most 2 re-prompts? Each one resends the conversation and costs a call; if two corrections fail, the problem is the prompt or the ticket, and a person reads it.
- Why SDK retries = 2 and no loop of ours? Network and rate-limit errors only; our own loop on top would multiply retries during an outage.
- Why few-shot and not zero-shot, which has one more right answer? One case is below what 30 cases can resolve; few-shot is 27% cheaper, 4 times faster at p90, and never re-prompts.
- Why not trust the judge for review? It matched our scores only 5 times out of 10 on that criterion.